

Lab questions 202606

Question 4 - 1x

Objective

Assess whether students understand how Unix privilege escalation can arise from misconfigured `sudo` rules, especially when environment variables such as `LD\_PRELOAD` or `LD\_LIBRARY\_PATH` are preserved.

The question evaluates the student's ability to distinguish between SetUID and sudo execution semantics, reason about dynamic linker behavior, identify exploitability conditions, and propose secure mitigations.

This is important because many real-world privilege escalations are not caused by complex memory corruption bugs, but by small configuration mistakes that allow an unprivileged user to influence privileged execution.

– FROM HERE

Scenario

During a post-exploitation assessment of a Linux server, you obtain a shell as the low-privileged user `deploy`.

The system is used by a small DevOps team. The user `deploy` is allowed to restart a custom backup helper through `sudo`, without being asked for a password.

Running `sudo -l` shows:

--- Begin ---

User deploy may run the following commands on backup01:

(root) NOPASSWD: /usr/local/bin/backup-check

--- End ---

You inspect the binary:

--- Begin ---

```
$ ls -l /usr/local/bin/backup-check
```

```
-rwxr-xr-x 1 root root 16840 May 10 09:12 /usr/local/bin/backup-check
```

```
$ file /usr/local/bin/backup-check
```

```
/usr/local/bin/backup-check: ELF 64-bit LSB pie executable, x86-64, dynamically linked
```

```
$ ldd /usr/local/bin/backup-check
```

```
linux-vdso.so.1
```

```
libbackup.so => /opt/backup/lib/libbackup.so
```

```
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
```

--- End ---

The sudoers configuration contains the following line:

--- Begin ---

```
Defaults env_keep += "LD_PRELOAD LD_LIBRARY_PATH"
```

--- End ---

Questions

Q1

Explain why the sudoers configuration is dangerous in this scenario.

Q2

Would the same LD_PRELOAD-based attack normally work against a SetUID-root binary? Explain why or why not.

Q3

Describe a practical exploitation strategy to obtain code execution as root using LD_PRELOAD.

You do not need to provide a full exploit, but your answer should include:

- what kind of shared object you would compile;
- what function or mechanism would trigger execution;
- how you would invoke the sudo-allowed binary.

Q4

Suppose LD_PRELOAD is removed from env_keep, but LD_LIBRARY_PATH is still preserved. The binary depends on libbackup.so.

Under what conditions could this still be exploitable?

Q5

Propose at least three defensive fixes. For each fix, briefly explain what risk it removes.

– TO HERE

Possible Answers

A1

The sudoers rule is dangerous because it preserves LD_PRELOAD and LD_LIBRARY_PATH when running a command as root. This lets deploy influence the dynamic linker and potentially load attacker-controlled code inside a root process.

A2

Normally, the same LD_PRELOAD trick would not work against a SetUID-root binary because the dynamic loader ignores dangerous environment variables in secure-execution mode.

With sudo, the risk reappears if sudoers explicitly preserves those variables.

A3

A valid exploitation strategy is to compile a malicious shared object and load it through LD_PRELOAD:

```
gcc -fPIC -shared -o /tmp/evil.so evil.c
```

```
sudo LD_PRELOAD=/tmp/evil.so /usr/local/bin/backup-check
```

The shared object can use a constructor function that runs automatically when the library is loaded, before main(), and spawns a root shell.

A4

If only LD_LIBRARY_PATH is preserved, exploitation may still be possible if the attacker can place a fake libbackup.so in a directory searched before the legitimate one.

Example:

```
sudo LD_LIBRARY_PATH=/tmp /usr/local/bin/backup-check
```

This works only if the dynamic loader resolves libbackup.so from /tmp instead of the trusted path.

A5

Good mitigations include removing LD_PRELOAD and LD_LIBRARY_PATH from env_keep, keeping env_reset enabled, avoiding unnecessary sudo access to dynamically linked helpers, ensuring privileged libraries are root-owned and not user-writable, and replacing broad sudo rules with minimal audited wrappers.

The core defense is to prevent unprivileged users from controlling the environment or filesystem inputs of root-executed programs.

Question 5 - 1.5x

Objective

Assess the student's understanding of the Fastbin dup technique, including the interaction between malloc(), free(), Fastbin linked lists, double-free protections, fake chunks, and the transition from an arbitrary write primitive to code execution. This is important because the slides explicitly show how seemingly simple memory-management bugs can be transformed into powerful exploitation primitives.

– FROM HERE

Scenario

A Linux service contains a double-free vulnerability on a heap object allocated with malloc(0x68).

During analysis, you determine that:

- The application uses a glibc version where Fastbins are present.
- The allocated chunk size falls within the Fastbin range.
- A target pointer named `auth_callback` exists in writable memory.
- The memory region surrounding `auth_callback` already contains data that can be interpreted as a valid fake Fastbin chunk of the appropriate size.
- ASLR is enabled.
- No stack overflow vulnerabilities are available.

Questions

1. Explain why a double free on a Fastbin-sized chunk may allow the same chunk to appear multiple times in a Fastbin list.
2. Directly freeing the same chunk twice may trigger a mitigation under some circumstances.
Why does the sequence:
`free(chunk_B);`
`free(chunk_A);`
`free(chunk_B);`
avoid the simplest double-free detection?
3. After allocating `chunk_B` again from the Fastbin, the attacker overwrites its `fd` pointer with the address of the fake chunk overlapping `auth_callback`.
Explain the role of the `fd` pointer in making the target memory eventually returned by `malloc()`.
4. Why must the target memory resemble a valid chunk, and what specific constraint discussed in the slides must be satisfied?
5. Assume the attacker successfully obtains a write primitive on `auth_callback`.
Compare the following two exploitation strategies:
 - Overwriting a function pointer directly.
 - Overwriting `__malloc_hook`.
6. What additional action is required to trigger execution when using `__malloc_hook`?
7. The slides discuss the role of ASLR in exploitation.
Explain why obtaining an arbitrary write primitive alone is often insufficient for reliable code execution when ASLR is enabled.

– TO HERE

Possible Answers

1. Fastbins are singly linked lists. A double free can cause the same chunk to be inserted multiple times, resulting in multiple references to the same memory region inside the Fastbin list.
2. The mitigation checks whether the chunk being freed is already at the Fastbin head. After freeing chunk_A, chunk_B is no longer at the head, so freeing chunk_B again bypasses this simple check.
3. malloc() removes chunks from the Fastbin head. By corrupting the fd pointer, the attacker controls the next pointer followed by malloc(), causing future allocations to return memory near the chosen target.
4. malloc() verifies the chunk size when allocating from a Fastbin. Therefore the fake chunk must contain a size value compatible with the Fastbin size being used.
5. A direct function-pointer overwrite executes when that pointer is invoked. For __malloc_hook, the attacker must trigger a subsequent malloc() call, which causes the hook function to be executed.
6. Call malloc(). The argument that needs to be used depends on the code that the attacker wants to execute. The parameter may or may not be irrelevant when using a one-gadget. In some cases the parameter could be a pointer to the string with the command to execute (instead of a size_t)
7. ASLR randomizes the locations of libc, the heap, the stack, and other mappings. Even with an arbitrary write primitive, the attacker typically needs a way to determine the address of useful code before reliably redirecting execution.